



Godot na prática: código, física e estados

Da teoria de nós e sinais para o código que faz o
personagem se mover, pular e reagir

Sumário



Fundamentos do código

Nós, cenas, GDScript, ciclo de vida, delta



Movimentação e física 2D

Vector2D, CharacterBody2D, gravidade, pulo, input



Machine State e States

Do if/else gigante à FSM organizada



Aplicação Prática

Mexendo no código + mecânica extra ao vivo





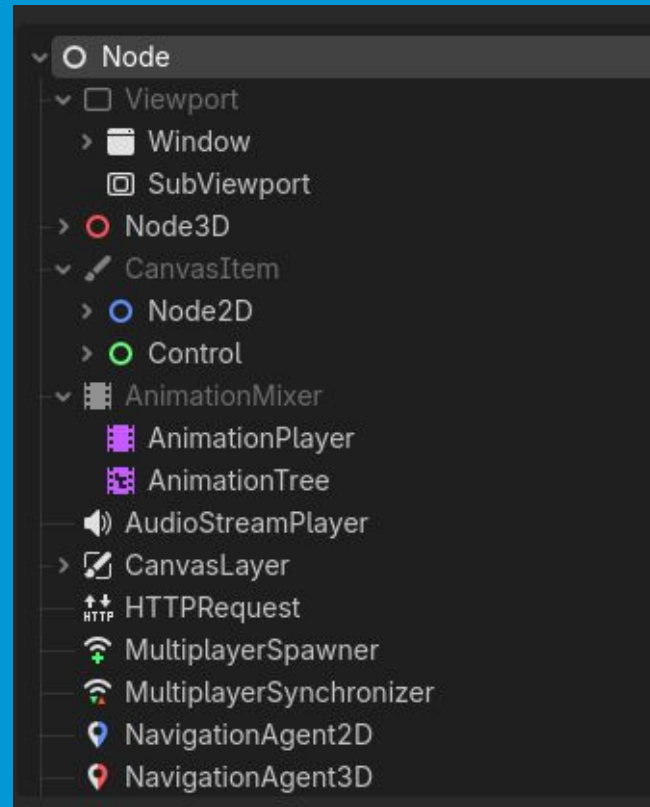
01

Fundamentos do Código

Como o Godot processa o código e
por que certas funções existem

O que é um Node?

- **Node (nó)** é a unidade básica de tudo no Godot.
- Cada nó tem um **tipo (classe)** que **define o que ele sabe fazer**.
- Nós são **organizados em uma árvore (Scene)** e todo nó tem um pai e pode ter vários filhos.
- A **posição de um nó na árvore importa**.



Tipos de Nodes



Node2D

Base de tudo que tem posição, rotação e escala no espaço 2D.



Sprite2D

Desenha uma imagem (textura) na tela. O visual do personagem.



CharacterBody2D

Corpo físico feito para personagens controlados por código (nosso player).



CollisionShape2D

Define a forma física (colisão) de um corpo - sempre filho de um body.



Area2D

Detecta sobreposição sem física real, ideal para gatilhos e coletáveis.

Tipos de Nodes



Control

Base para elementos de interface (UI): botões, labels, painéis.



AnimationPlayer

Controla animações, troca de sprites, movimento de propriedades.



Node

A base de todos os outros nós. Não tem posição nem visual, somente lógica, ciclo de vida e lugar na árvore.



AudioStreamPlayer

Serve para reproduzir sons e músicas de forma não posicional.

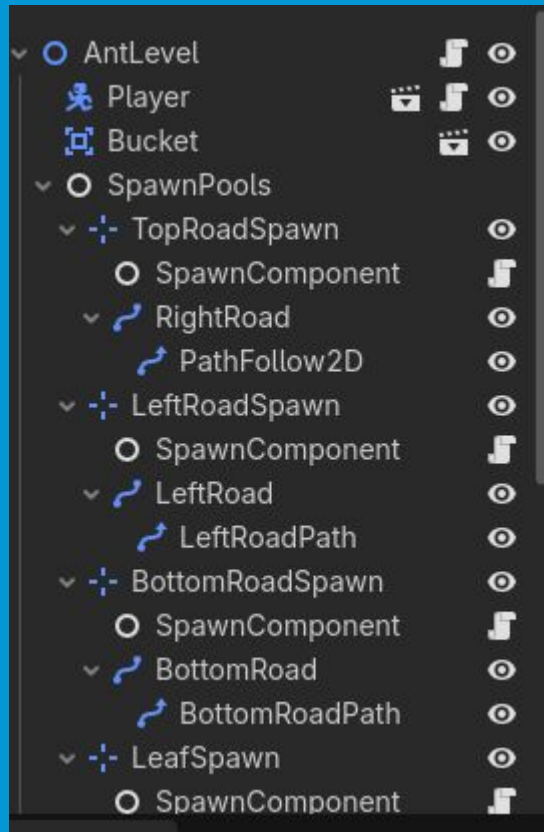


TileMap/TileMapLayer

Monta o cenário (chão, plataformas) a partir de uma grade de tiles.

E as Cenas (Scenes)?

- Uma **cena (.tscn)** é uma árvore de nós salva.
- **Todo projeto Godot é, na prática, um conjunto de cenas conectadas.**
- Uma cena sempre tem um **nó raiz** e é ele que **dá nome e tipo** à cena inteira.
- Cenas podem ser **instanciadas dentro de outras cenas** (como um componente reutilizável).



GDScript - A Linguagem do Godot

- **GDScript (.gd)** é a linguagem nativa da **Godot**: sintaxe inspirada em Python, tipagem opcional (mas recomendada), e **integração direta com o editor**, sem compilação separada.

```
# Primitivos
var nome: String = "Ana"
var vida: int = 100
var velocidade: float = 4.5
var ativo: bool = true

# Vector2 / Vector3 – nativos, usados o tempo todo
var posicao: Vector2 = Vector2(100, 200)
```

```
# Array – pode ser tipado
var inimigos: Array[String] = ["Goblin", "Orc"]

# Dictionary – como um objeto/dict do Python
var item: Dictionary = {
    "nome": "Espada",
    "dano": 15
}

# Constante
const GRAVIDADE: float = 980.0
```




GDScript - A Linguagem do Godot

- Assim como existem as **Nodes** e as **Scenes**, os arquivos que fazemos os códigos em **GDScript** são chamados de **Scripts**.
- E esses **Scripts** viram o “cérebro” de um **nó específico** quando você anexa esse script a ele.
 - A primeira linha declara de qual tipo de nó ele estende.

```
Depuração script_teste.gd
1 extends Node
2
3
4 # Called when the node enters the scene tree for the first time.
5 func _ready() -> void:
6     pass # Replace with function body.
7
8
9 # Called every frame. 'delta' is the elapsed time since the previous frame.
10 func _process(delta: float) -> void:
11     pass
```

```
Node2D
extends Node2D

# Called when the node enters the scene tree for the first time.
func _ready() -> void:
    pass # Replace with function body.
```





GDScript - Funções Nativas



`_ready()`

Chamada uma vez, quando a Node entra na árvore da cena, ideal para **inicialização**.



`_input(event)`

Chamada sempre que **ocorre um evento** de entrada do jogador (tecla, mouse).



`_process(delta)`

Chamada a cada frame renderizado. Recebe o tempo decorrido (delta) desde o último frame. **Usado para UI.**



`_physics_process(delta)`

Chamada em intervalo fixo. Use para **movimento, colisão e física**, mais previsível que `_process`.



O que é o delta?

- **Delta** é o tempo (em segundos) que passou desde o último frame/chamada. É geralmente um **número bem pequeno**, tipo 0.016 (a 60 FPS).
1. Se você move o personagem com um **valor fixo por frame**, o jogo fica **mais rápido em PCs potentes e mais lento em PCs fracos**.

Ex: `position.x += 5`

2. **Multiplicando por delta**, o movimento vira “unidades por segundo”, não “unidades por frame”, ou seja, **o jogo fica igual em qualquer máquina**.

Ex: `position.x += velocidade * delta`



Acessando Nodes

- Para um script mexer em outro nó (um filho, um irmão), ele precisa de uma referência para esse nó.
- Um script pode acessar outros Nodes da árvore: pelo caminho (\$), por busca (get_node) ou por grupo (get_tree().get_nodes_in_group()).

```
extends CharacterBody2D

# @onready garante que o node já existe na árvore quando roda
@onready var sprite: Sprite2D = $Sprite2D
@onready var sprite2: Sprite2D = get_node("Sprite2D")

# Caminho relativo mais longo
@onready var hitbox: Area2D = $Hitbox/CollisionShape2D

func _ready() -> void:
    sprite.play("idle")
```





Sinais (Signals)

- **Sinais** são o sistema de eventos da Godot, equivalente a eventos customizados. Um **Node emite um sinal**, outros **Nodes escutam**, sem **acoplamento direto** entre eles.
- **Prefira sinais** a chamar métodos diretamente entre scripts distantes na árvore, assim o **Player não precisa "saber" que a HUD existe**.

```
extends CharacterBody2D

# Declaração do sinal - pode ter parâmetros tipados
signal health_changed(new_health: int, max_health: int)
signal died

var max_health: int = 3
var current_health: int = max_health

func take_damage(amount: int) -> void:
    current_health = max(current_health - amount, 0)

    # Emite o sinal - Godot usa .emit(), avisando quem estiver escutando
    health_changed.emit(current_health, max_health)

    if current_health <= 0:
        died.emit()
```

```
extends CanvasLayer

@onready var health_label: Label = $HealthLabel

func _ready() -> void:
    var player := get_tree().get_first_node_in_group("player")

    # Conecta o sinal do Player a uma função aqui na HUD
    player.health_changed.connect(_on_player_health_changed)
    player.died.connect(_on_player_died)

func _on_player_health_changed(new_health: int, max_health: int) -> void:
    health_label.text = "Vida: %d/%d" % [new_health, max_health]

func _on_player_died() -> void:
    health_label.text = "Game Over"
```

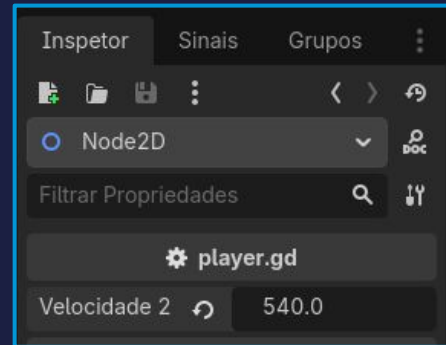
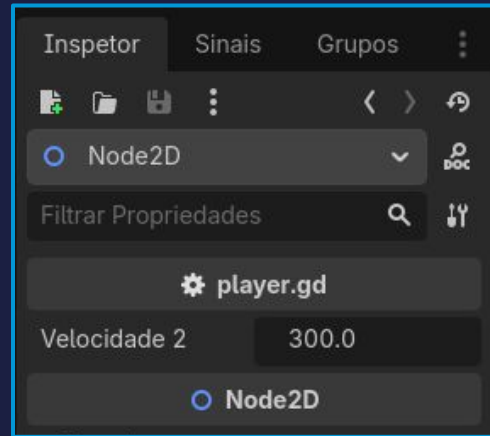


@export: expondo variáveis pro Inspector

- **@export** transforma uma **variável** do **script** em um **campo editável** direto no painel **Inspetor do Godot**, sem precisar mexer em código.
- Muito usado para **valores de teste**: **velocidade**, **força do pulo**, **gravidade**, **vida**, **dano** e etc...

```
extends Node2D

var velocidade: float = 300.0 # só no código
@export var velocidade2: float = 300.0 # editável no Inspetor
```

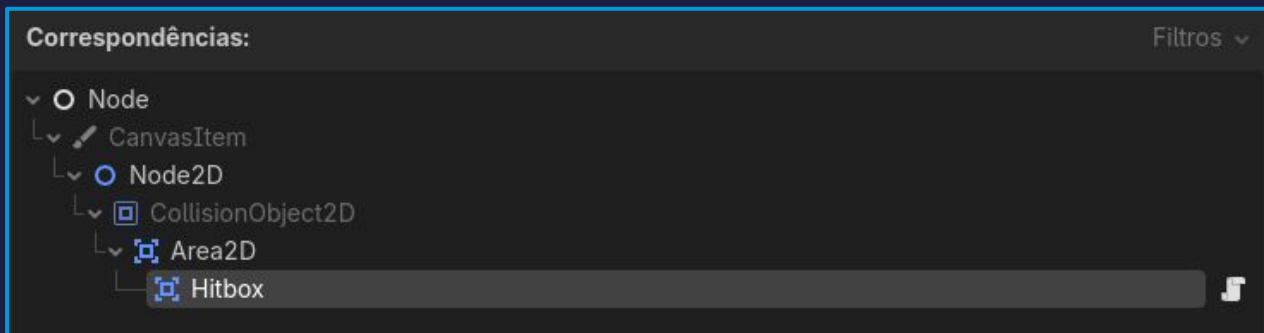


class_name

- Você pode criar tipos próprios estendendo Nodes.
- Para fazer isso, você só precisa colocar um **class_name** NomeDaClasse antes do **extends** do Node.

```
# Hitbox.gd
class_name Hitbox
extends Area2D

@export var damage: int = 10
```



class_name

- **Acesso Global:** Torna o script acessível em qualquer outro lugar do projeto sem precisar usar o caminho do arquivo.
- **Autocompletar:** Melhora as sugestões de código no editor.
- **Herança Simples:** Facilita estender o script em outros arquivos usando apenas `extends NomeDaClasse`.
- **Editor de Nós:** Adiciona a classe diretamente na lista de nós do editor, permitindo instanciá-la facilmente.

```
# Player.gd
extends CharacterBody2D

# 1. Checagem de Tipo / Autocompletar:
func _on_area_entered(area: Area2D) -> void:
    if area is Hitbox: # Hitbox é reconhecido globalmente
        take_damage(area.damage)

# 2. Tipagem forte de variáveis:
var current_hitbox: Hitbox
```




Movimentação e Física 2D



Mostrar na prática a física de um jogo de plataforma



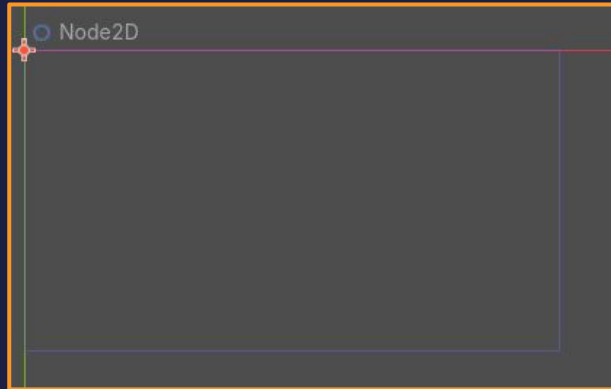
02



Plano 2D do Godot



- **Eixo X:** igual ao que todo mundo já conhece, **direita é positivo e esquerda é negativo**.
- **Eixo Y: INVERTIDO** em relação à matemática “normal”, para **BAIXO** é positivo e para **CIMA** é negativo.



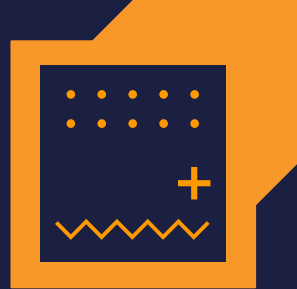
Vector2



- Um **Vector2** guarda dois números **(x, y)**, mas o que ele representa muda com o contexto.
- **position** → onde o nó está no espaço
- **velocity** → pra onde e com que intensidade algo está se movendo
- **direction** → só a direção (sem se importar com intensidade)

```
var posicao: Vector2 = Vector2(100, 200) # Onde estou
var velocidade: Vector2 = Vector2(200, 0) # Pra onde vou + velocidade
var direcao: Vector2 = Vector2(1, 0) # Apenas a direção (normalized)
```

Manipulando Vector2



- **Mover/somar posições:** Move um objeto no espaço ou aplica uma força.

position += Vector2(10, 0)

- **Alterar a intensidade:** Aumenta ou diminui a velocidade mantendo a mesma posição.

velocity = direction * speed

- **Normalizar o vetor (.normalized()):** Garante que o vetor tenha tamanho igual a 1.

direction = direction.normalized()



Tipos de Body2D

Tipo de corpo	Quando usar
CharacterBody2D	Controlado 100% por código (nosso player, inimigos)
RigidBody2D	Física realista automática (caixas, objetos que quicam/caem)
StaticBody2D	Nunca se move (chão, paredes, plataformas fixas)

- >  StaticBody2D
-  CharacterBody2D
-  RigidBody2D



move_and_slide()



- **CharacterBody2D** já vem com a propriedade **velocity** e o método **move_and_slide()**, que resolve colisões com paredes e chão automaticamente.

```
extends CharacterBody2D

func _physics_process(delta: float) -> void:
    # Alteramos a variável interna velocity
    velocity.x = 200.0

    # Chamamos o método para aplicar o movimento e tratar colisões
    move_and_slide()

    # Agora podemos consultar o estado do personagem!
    if is_on_floor():
        print("Estou tocando no chão!")
```





Mecânicas Icônicas de Plataforma



- **Gravidade:** soma peso quando o personagem **não** está no chão.
- **Pulo:** troca instantaneamente a **velocidade vertical**.

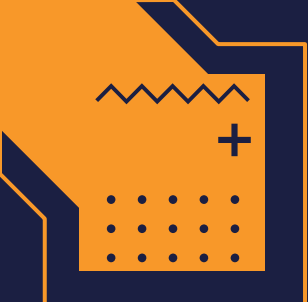
```
const GRAVIDADE := 980.0
const FORCA_PULO := -400.0 # negativo = pra cima!

func _physics_process(delta: float) -> void:
    if not is_on_floor():
        velocity.y += GRAVIDADE * delta # cai

    if Input.is_action_just_pressed("pular") and is_on_floor():
        velocity.y = FORCA_PULO # pula

    move_and_slide()
```





Input.get_vector() e move_toward()



- O **get_vector** serve para movimentação 2D (2 eixos).

`Input.get_vector("negativo_x", "positivo_x", "negativo_y", "positivo_y")`

- **move_toward()** suaviza aceleração e desaceleração.

```
# Reduz a velocidade horizontal (velocity.x) em direção a 0,  
# subtraindo/somando até 100.0 por segundo (se multiplicado por delta)  
body.velocity.x = move_toward(body.velocity.x, 0, desaceleration * delta)
```



```
body.velocity.x = move_toward(  
    body.velocity.x,  
    target_speed,  
    acceleration * delta  
)
```





Machine State e States

Boas práticas de código para
organizar comportamentos

03



Código Gigante de Ifs e Booleanos

```
var is_jumping := false
var is_running := false
var is_attacking := false
var is_crouching := false

func _physics_process(delta):
    if is_jumping and not is_attacking:
        ...
    elif is_running and not is_crouching and not is_attacking:
        ...
    # a cada mecânica nova, isso PIORA exponencialmente
```

- É assim que a maioria dos projetos começa e, é assim que eles ficam ilegíveis rapidamente.



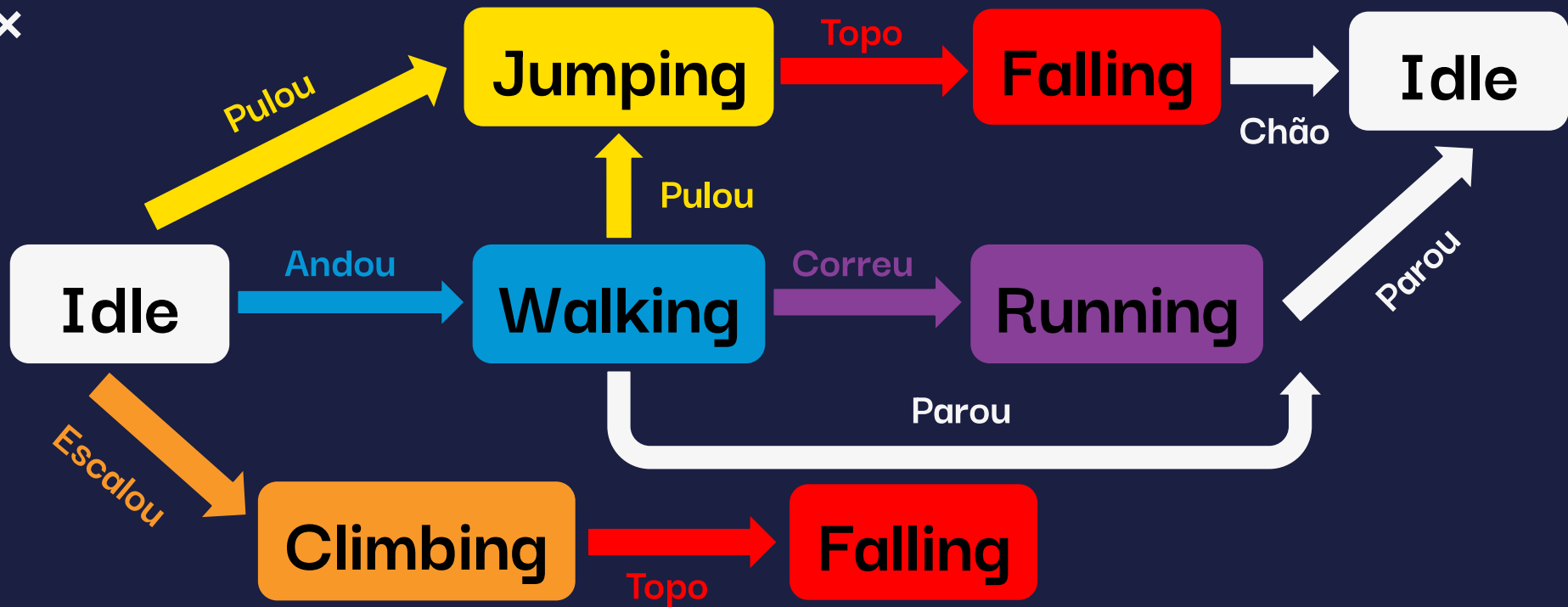


Máquina de Estados (FSM)

- **Estados:** Idle, Walking, Running, Jumping, Falling, Climbing...
- **Máquina de Estados Finita:** é uma máquina que permite que algo (no nosso caso, o player) esteja sempre em **exatamente um estado por vez**.
 - **Estado** sabe o que **fazer enquanto está ativo** e **quando deve transicionar para outro estado**.
 - **Transições** acontecem **por eventos claros**.



Mapa de Estados do Player





State Machine como Nós

- Quando o jogo cresce, **cada estado vira seu próprio nó**, mais organizado e fácil de estender.
- **StateMachine(gerenciador)**: guarda o **current_state** e delega as chamadas (**physics_update**, **_change_state** e **transition_to**) para ele.
- **State (classe base)**: define o “contrato” que todo estado segue - **enter()**, **exit()**, **physics_update()**
 - **Estados concretos**: `IdleState.gd`, `RunningState.gd`, `JumpingState.gd`, cada um só cuida da própria lógica.



Aplicação Prática

Construir uma máquina de
estados com seus estados

04



Passo a Passo

1. Adicionar as ações “jump” e “run”;
2. Adicionar elas no **InputComponent**;
3. Preparar o **MovementComponent** para ser comandado por estados;
4. Criar **state.gd** e **state_machine.gd**;
5. Criar os estados: **Idle**, **Walking**, **Jumping**, **Falling**, **Running** e **Climbing**;
 - a. Setar **initial_state = Idle** no Inspector.
6. Adaptar o **player.gd** e adicionar na cena do jogador (**player.tscn**) a **StateMachine** com seus Estados.



Referências Bibliográficas



- <https://docs.godotengine.org/en/stable/>
- <https://docs.godotengine.org/pt-br/4.x/>

